

Software Engineering Project May-2026

MILESTONE 2 DELIVERABLES

LearnAtHome

Submitted by

Team Name: Synergy

Team Code: 049



IITM Online BS Degree Program,
Indian Institute of Technology, Madras, Chennai
Tamil Nadu, India, 600036

HOME TUITION PLATFORM

Project Details

Software Name : **LearnAtHome**

Domain : **Home Tuition Platform**

Date of Submission : **23-07-2026**

Team Details

Team Name : **Synergy**

Team Code : **Team-049**

Name	Roll No.	Roles
Sreelakshmy J Menon	21f1002459	Product manager
Drishya Shrivastava	22f2000244	Scrum Master
Medhavi Tripathi	22f3002491	Tester
Sudarshini V	22f3000519	Frontend Developer
Shikha Rani	24f2007655	Backend Developer

Tasks Assigned - Milestone 2

Name	Roll No.	Preferred Roles	TASKS assigned
Sreelakshmy J Menon	21f1002459	Product Manager	● Project scheduling for Milestone 2 ● Class Diagram ● Review and Documentation of Milestone 2
Drishya Shrivastava	22f2000244	Scrum Master	● Login/Register Page ● App Home Page ● Prepare Sprint Schedule/Meetings/ Gantt Chart ● Formatting Milestone Document ● Maintain MoM document ● Send required reports to TA
Medhavi Tripathi	22f3002491	Tester	● Database Design ● ER Diagram ● Parent Dashboard ● Formatting & Documenting Milestone Document ● Testing
Sudarshini V	22f3000519	Frontend Developer	● Prototyping Dashboards ● Admin Dashboard ● Student Dashboard
Shikha Rani	24f2007655	Backend Developer	● Design of Components ● Tutor Dashboard ● Wireframe

Component Design

→ Tutor dashboard frontend

- ◆ Story 1.1 : In-App Session Update Notification
- ◆ Story 2.4: Per-Student Learning Pace Log
- ◆ Story 3.1: Unified Class Schedule Management
- ◆ Story 3.3 : Virtual Meeting Request

→ Parent dashboard design

- ◆ Story 1.2: Two-Way Parent-Tutor Messaging
- ◆ Story 2.1: Student Learning Progress Dashboard
- ◆ Story 5.3: Weekly Session Summary Notification
- ◆ Story 4.2: Monthly Curriculum Plan for Parents

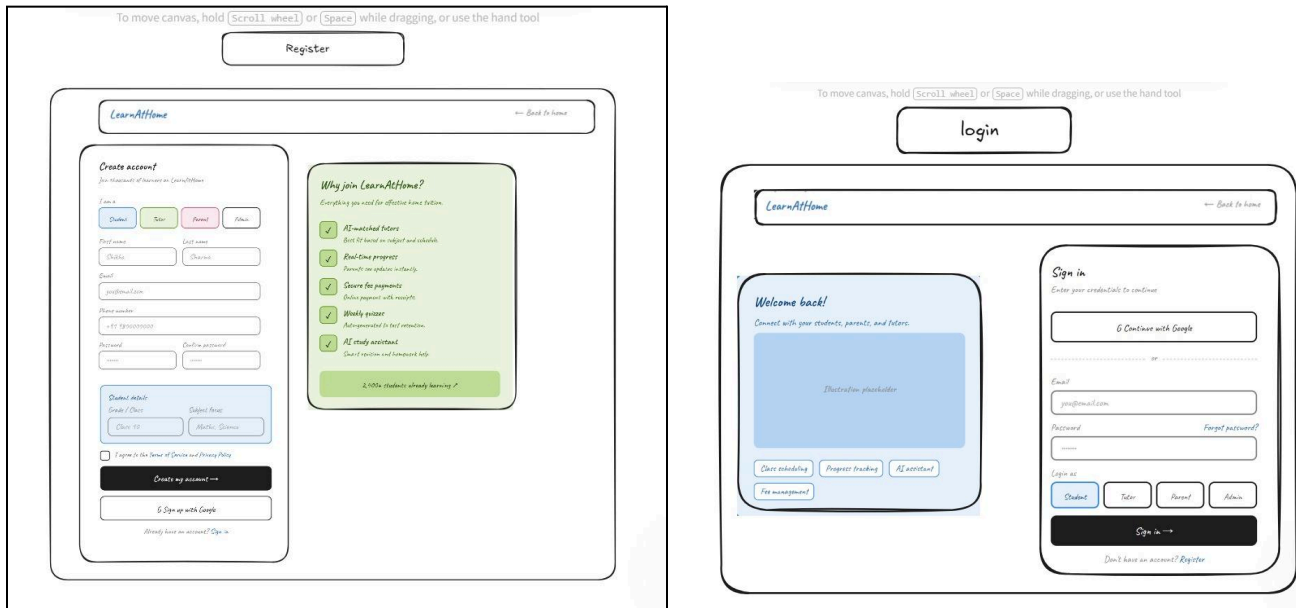
→ Student dashboard design

- ◆ Story 1.3 FAQ Board
- ◆ Story 2.2 : Visual Progress Tracker for Students
- ◆ Story 2.3: Weekly Quiz for Weakness Identification
- ◆ Story 3.2: Flexible Session Booking
- ◆ Story 3.4: One to One Session Booking
- ◆ Story 4.1: Upcoming Session Details
- ◆ Story 4.3: Study Tips per Session
- ◆ Story 5.1: Interactive Assignments
- ◆ Story 5.2: Curated Study Resources per Topic

Architecture Style

- ★ Modular monolith with clear domain separation
- ★ Flask backend with role based frontend and shared database

Wireframes



Database Design Approach

1. Database Design Approach

The LearnAtHome database has been designed using a user story-driven approach. Every table and relationship has been derived from the finalized functional requirements to ensure that the database accurately represents the business processes of the application.

The following principles were followed during the design process:

- User story-driven database design
- Normalized relational database structure
- Minimal data redundancy
- Clear separation of functional modules
- Scalable and maintainable architecture
- Consistent naming conventions

2. Module 1 : User Management

The User Management module stores information related to all users of the LearnAtHome platform. It supports authentication, user identification, and role-based access control for administrators, tutors, parents, and students.

Table 2.1 – Roles

Purpose

Stores the different user roles available within the LearnAtHome platform. Each user references a role through role_id, ensuring consistency and reducing redundancy across the database.

Supported User Stories

- Supports all user stories by enabling role-based access for different users of the system.

Relationships

- One Role → Many Admins
- One Role → Many Tutors
- One Role → Many Parents
- One Role → Many Students

Attributes

Column	Data Type	Key	Description
role_id	INT	PK	Unique role identifier
role_name	VARCHAR(20)	UNIQUE	Name of the role (Admin, Tutor, Parent, Student)

Design Justification

Maintaining a separate Roles table eliminates repeated storage of role names across user tables and provides a scalable approach for role management.

Table 2.2 – Admin

Purpose

Stores administrator information responsible for managing the LearnAtHome platform and maintaining system-level operations.

Supported User Stories

- Indirectly supports all user stories by managing platform users and content.

Relationships

- One Role → Many Admins

Attributes

Column	Data Type	Key	Description
admin_id	INT	PK	Unique administrator identifier
role_id	INT	FK → Roles(role_id)	Associated role
username	VARCHAR(50)	UNIQUE	Administrator username
password_hash	VARCHAR(255)		Encrypted password
registered_at	DATETIME		Account registration date and time
last_login_at	DATETIME		Most recent login date and time

Design Justification

Administrators perform system management tasks that are distinct from tutors, parents, and students. A dedicated table simplifies role-specific access control.

Table 2.3 – Tutor

Purpose

Stores information about the tutor responsible for conducting sessions, preparing assessments, tracking student progress, and communicating with students and parents.

Supported User Stories

- Story 1.1
- Story 2.4
- Story 3.1
- Story 3.3
- Story 4.2
- Story 5.3

Relationships

- One Tutor → Many Sessions
- One Tutor → Many TeachingPlans
- One Tutor → Many Quizzes
- One Tutor → Many MeetingRequests

Attributes

Column	Data Type	Key	Description
tutor_id	INT	PK	Unique tutor identifier
role_id	INT	FK → Roles(role_id)	Associated role
tutor_name	VARCHAR(100)		Tutor's full name
email	VARCHAR(100)	UNIQUE	Login email
phone_no	VARCHAR(15)		Contact number
experience_years	INT		Teaching experience
password_hash	VARCHAR(255)		Encrypted password
is_active	BOOLEAN		Account status
registered_at	DATETIME		Account registration date and time
last_login_at	DATETIME		Most recent login date and time

Design Justification

Although the current system contains only one tutor, maintaining a dedicated Tutor table supports future scalability and keeps tutor-specific information separate from other user roles.

Table 2.4 – Parent

Purpose

Stores information about parents registered on the LearnAtHome platform. Parents can monitor their child's academic progress, receive session updates, communicate with the tutor, and view weekly summaries.

Supported User Stories

- Story 1.1 – In-App Session Update Notification
- Story 1.2 – Two-Way Parent-Tutor Messaging
- Story 2.1 – Student Learning Progress Dashboard
- Story 4.1 – Upcoming Session Details
- Story 5.3 – Weekly Summary Notification

Relationships

- One Role → Many Parents
- One Parent → Many Students
- One Parent → Many Meeting Requests
- One Parent → Many Notifications

Attributes

Column	Data Type	Key	Description
parent_id	INT	PK	Unique parent identifier
role_id	INT	FK → Roles(role_id)	Associated role
parent_name	VARCHAR(100)		Parent's full name
email	VARCHAR(100)	UNIQUE	Login email
phone_no	VARCHAR(15)		Contact number
password_hash	VARCHAR(255)		Encrypted password
is_active	BOOLEAN		Account status
registered_at	DATETIME		Account registration date and time
last_login_at	DATETIME		Most recent login date and time

Design Justification

Parents have responsibilities that differ from tutors and students. Maintaining a separate Parent table enables role-based access while allowing each parent to be associated with one or more students.

Table 2.5 – Student**Purpose**

Stores information about students enrolled on the LearnAtHome platform. Students are the primary users of the system and participate in tuition sessions, assignments, quizzes, study activities, and progress tracking.

Supported User Stories

- Supports all student-related user stories throughout the platform.

Relationships

- One Role → Many Students
- One Parent → Many Students
- One Student → Many Session Bookings
- One Student → Many Assignment Submissions
- One Student → Many Quiz Attempts
- One Student → Many Learning Progress Records
- One Student → Many Study Tips
- One Student → Many Notifications

Attributes

Column	Data Type	Key	Description
student_id	INT	PK	Unique student identifier
role_id	INT	FK → Roles(role_id)	Associated role
parent_id	INT	FK → Parent(parent_id)	Parent of the student
student_name	VARCHAR(100)		Student's full name
email	VARCHAR(100)	UNIQUE	Login email
phone_no	VARCHAR(15)		Contact number
school	VARCHAR(100)		School currently attending
password_hash	VARCHAR(255)		Encrypted password
is_active	BOOLEAN		Account status
registered_at	DATETIME		Account registration date and time
last_login_at	DATETIME		Most recent login date and time

Design Justification

The Student table serves as the central user entity of the LearnAtHome platform. It is referenced by multiple academic and communication modules, making it essential to maintain student information separately from other user roles.

3. Module 2 – Academic Management

The Academic Management module manages the core teaching activities of the LearnAtHome platform. It maintains information related to subjects, student enrollments, tuition sessions, and session updates. These entities form the foundation upon which assignments, quizzes, progress tracking, and learning resources are built.

Table 3.1 – Subject

Purpose

Stores all subjects offered by the tutor. Each subject can have multiple enrolled students, and each student can enroll in multiple subjects.

Supported User Stories

- Story 3.2 – Flexible Session Booking
- Story 4.2 – Monthly Teaching Plan
- Story 5.2 – Curated Study Resources

Relationships

- One Subject → Many StudentSubject Records
- One Subject → Many Sessions
- One Subject → Many Teaching Plans
- One Subject → Many Quizzes

Attributes

Column	Data Type	Key	Description
subject_id	INT	PK	Unique subject identifier
subject_name	VARCHAR(50)	UNIQUE	Name of the subject

Design Justification

A separate Subject table eliminates repeated storage of subject names throughout the database and supports efficient management of academic content.

Table 3.2 – StudentSubject

Purpose

Represents student enrollment in subjects. This table resolves the many-to-many relationship between students and subjects.

Supported User Stories

- Story 3.2 – Flexible Session Booking
- Story 4.2 – Monthly Teaching Plan
- Story 5.2 – Curated Study Resources

Relationships

- One Student → Many StudentSubject Records
- One Subject → Many StudentSubject Records

Attributes

Column	Data Type	Key	Description
student_subject_id	INT	PK	Unique enrollment identifier
student_id	INT	FK → Student(student_id)	Enrolled student
subject_id	INT	FK → Subject(subject_id)	Selected subject

Design Justification

A student may enroll in multiple subjects, while each subject may have multiple students. The StudentSubject table resolves this many-to-many relationship while maintaining database normalization. Each student can enroll in a particular subject only once. Therefore, the combination of **student_id** and **subject_id** should remain unique.

Table 3.3 – Session

Purpose

Stores all tuition sessions conducted by the tutor. Every session belongs to exactly one subject and serves as the central academic entity for assignments, study resources, learning progress, and session updates.

Supported User Stories

- Story 1.1 – In-App Session Update Notification
- Story 2.4 – Learning Pace Log
- Story 3.1 – Unified Class Schedule Management
- Story 3.2 – Flexible Session Booking

- Story 3.3 – Virtual Meeting Request
- Story 3.4 – One-to-One Session Booking
- Story 4.1 – Upcoming Session Details

Relationships

- One Tutor → Many Sessions
- One Subject → Many Sessions
- One Session → Many Session Bookings
- One Session → Many Assignments
- One Session → Many Learning Progress Records
- One Session → Many Study Tips
- One Session → Many Study Resources

Attributes

Column	Data Type	Key	Description
session_id	INT	PK	Unique session identifier
tutor_id	INT	FK → Tutor(tutor_id)	Tutor conducting the session
subject_id	INT	FK → Subject(subject_id)	Subject taught during the session
session_date	DATE		Scheduled session date
start_time	TIME		Session start time
end_time	TIME		Session end time
session_type	ENUM		Type of tuition session (Regular, One-to-One)
status	ENUM		Scheduled, Completed, Cancelled, Rescheduled

Design Justification

The Session table is the core entity of the LearnAtHome platform. Most academic activities are directly associated with a session, making it the central point of reference for teaching and learning.

Table 3.4 – SessionUpdate**Purpose**

Stores the summary of each completed tuition session, including topics covered, homework assigned, and the next scheduled session. This information is shared with students and parents after every class.

Supported User Stories

- Story 1.1 – In-App Session Update Notification

Relationships

- One Session → One Session Update

Attributes

Column	Data Type	Key	Description
update_id	INT	PK	Unique session update identifier
session_id	INT	FK → Session(session_id)	Associated session
topics_covered	TEXT		Topics discussed during the session
homework_assigned	TEXT		Homework assigned after the session
next_session_date	DATE		Planned date of the next session
notification_time	DATETIME		Timestamp when the update was sent

Design Justification

Separating session updates from the Session table prevents academic summaries from cluttering scheduling information while enabling structured communication after every completed class. Since each completed session has exactly one summary, every session can have only one associated SessionUpdate record.

4. Module 3 – Session Activities

The Session Activities module manages all academic activities that take place during or after a tuition session. It includes assignments, learning progress tracking, personalized study tips, curated study resources, and teaching plans. Since these activities are directly related to teaching and learning, most of the tables in this module are associated with the **Session** entity.

Table 4.1– Assignment

Purpose

Stores assignments created by the tutor after a tuition session. Assignments reinforce concepts taught during class and provide students with additional practice.

Supported User Stories

- Story 5.1 – Interactive Assignments

Relationships

- One Session → Many Assignments
- One Assignment → Many Assignment Submissions

Attributes

Column	Data Type	Key	Description
assignment_id	INT	PK	Unique assignment identifier
session_id	INT	FK → Session(session_id)	Session in which the assignment was given
title	VARCHAR(150)		Assignment title
description	TEXT		Assignment instructions
due_date	DATE		Assignment submission deadline

Design Justification

Assignments are linked to sessions because they are created based on the topics covered during class. Separating assignments from submissions allows multiple students to complete the same assignment without duplicating assignment details.

Table 4.2 – AssignmentSubmission**Purpose**

Stores assignment submissions made by students. Each record represents an individual student's submission for a particular assignment.

Supported User Stories

- Story 5.1 – Interactive Assignments

Relationships

- One Assignment → Many Assignment Submissions
- One Student → Many Assignment Submissions

Attributes

Column	Data Type	Key	Description
submission_id	INT	PK	Unique submission identifier
assignment_id	INT	FK → Assignment(assignment_id)	Assignment being submitted
student_id	INT	FK → Student(student_id)	Student submitting the assignment
submission_date	DATETIME		Date and time of submission
status	ENUM		Pending, Submitted, Late
tutor_feedback	TEXT		Feedback provided by the tutor

Design Justification

Separating assignment submissions from assignments enables multiple students to submit the same assignment independently while allowing the tutor to provide personalized feedback. A student may submit a particular assignment only once. Therefore, each combination of **assignment_id** and **student_id** must remain unique.

Table 4.3 – LearningProgress**Purpose**

Stores the tutor's evaluation of each student's learning progress after a completed session. The table records the student's overall learning status, learning pace, and tutor remarks for that session.

Supported User Stories

- Story 2.1 – Student Learning Progress Dashboard
- Story 2.2 – Visual Progress Tracker
- Story 2.4 – Per-Student Learning Pace Log

Relationships

- One Session → Many Learning Progress Records
- One Student → Many Learning Progress Records

Attributes

Column	Data Type	Key	Description
progress_id	INT	PK	Unique progress record
session_id	INT	FK → Session(session_id)	Session being evaluated
student_id	INT	FK → Student(student_id)	Student being evaluated
session_completion_status	ENUM		Completed, Partially Completed, Needs Revision
learning_pace	ENUM		Fast, Average, Needs Practice
tutor_remarks	TEXT		Tutor's observations for the session

Design Justification

Learning progress is maintained separately to provide a structured record of each student's performance over multiple sessions. Associating progress with a session preserves the context in which the evaluation was made while supporting parent and student dashboards.

Table 4.4 – StudyTip**Purpose**

Stores personalized study tips shared by the tutor after a session to help students improve their learning and study habits.

Supported User Stories

- Story 4.3 – Study Tips per Session

Relationships

- One Session → Many Study Tips
- One Student → Many Study Tips

Attributes

Column	Data Type	Key	Description
tip_id	INT	PK	Unique study tip identifier
session_id	INT	FK → Session(session_id)	Associated session
student_id	INT	FK → Student(student_id)	Student receiving the tip
tip_text	TEXT		Personalized study recommendation
created_at	DATETIME		Time when the tip was created

Design Justification

Study tips are stored independently from learning progress because they represent actionable recommendations rather than academic evaluations.

Table 4.5 – StudyResource**Purpose**

Stores learning resources recommended by the tutor for concepts covered during a session. Resources may include notes, videos, websites, PDFs, or practice worksheets.

Supported User Stories

- Story 5.2 – Curated Study Resources

Relationships

- One Session → Many Study Resources

Attributes

Column	Data Type	Key	Description
resource_id	INT	PK	Unique resource identifier
session_id	INT	FK → Session(session_id)	Session associated with the resource
resource_title	VARCHAR(150))		Title of the resource
resource_type	ENUM		PDF, Video, Website, Notes, Practice Sheet
resource_link	VARCHAR(255))		URL or file path of the resource

Design Justification

Learning resources are linked to sessions to ensure that students receive materials relevant to the concepts taught during that particular class.

Table 4.6 – TeachingPlan

Purpose

Stores the tutor's monthly teaching plan for each subject. It enables parents to view upcoming topics and helps the tutor organize lesson schedules.

Supported User Stories

- Story 4.2 – Monthly Teaching Plan

Relationships

- One Tutor → Many Teaching Plans
- One Subject → Many Teaching Plans

Attributes

Column	Data Type	Key	Description
plan_id	INT	PK	Unique teaching plan identifier
tutor_id	INT	FK → Tutor(tutor_id)	Tutor creating the teaching plan

subject_id	INT	FK → Subject(subject_id)	Subject covered by the plan
month	VARCHAR(20)		Month of the teaching plan
topic_name	VARCHAR(100)		Planned topic
planned_date	DATE		Expected teaching date

Design Justification

The TeachingPlan table provides a structured monthly roadmap of lessons for each subject. Separating it from sessions allows long-term planning while supporting parent visibility into upcoming learning objectives.

5. Module 4 – Assessment

The Assessment module manages student evaluation through weekly quizzes and performance summaries. It enables the tutor to assess student understanding, identify learning gaps, and communicate overall weekly progress to parents.

Table 5.1 – Quiz

Purpose

Stores weekly quizzes created by the tutor to assess students' understanding of a specific subject. Each quiz belongs to one subject and can be attempted by multiple students.

Supported User Stories

- Story 2.3 – Weekly Quiz for Weakness Identification

Relationships

- One Tutor → Many Quizzes
- One Subject → Many Quizzes
- One Quiz → Many Quiz Questions
- One Quiz → Many Quiz Attempts

Attributes

Column	Data Type	Key	Description
quiz_id	INT	PK	Unique quiz identifier
tutor_id	INT	FK → Tutor(tutor_id)	Tutor creating the quiz

subject_id	INT	FK → Subject(subject_id)	Subject being assessed
title	VARCHAR(100)		Quiz title
week_number	INT		Academic week number
created_at	DATETIME		Quiz creation timestamp

Design Justification

Each quiz is associated with one subject to simplify assessment, reporting, and progress analysis while supporting weekly evaluations.

Table 5.2 – QuizQuestion

Purpose

Stores all questions that belong to a quiz. Each quiz consists of multiple questions.

Supported User Stories

- Story 2.3 – Weekly Quiz for Weakness Identification

Relationships

- One Quiz → Many Quiz Questions

Attributes

Column	Data Type	Key	Description
question_id	INT	PK	Unique question identifier
quiz_id	INT	FK → Quiz(quiz_id)	Associated quiz
question	TEXT		Question statement
option_a	VARCHAR(255)		First option
option_b	VARCHAR(255)		Second option
option_c	VARCHAR(255)		Third option
option_d	VARCHAR(255)		Fourth option
correct_option	CHAR(1)		Correct answer (A, B, C or D)

Design Justification

Separating quiz questions from quizzes eliminates duplication and allows each quiz to contain any number of questions.

Table 5.3 – QuizAttempt

Purpose

Stores each student's quiz attempt and the score obtained.

Supported User Stories

- Story 2.1 – Student Learning Progress Dashboard
- Story 2.2 – Visual Progress Tracker
- Story 2.3 – Weekly Quiz for Weakness Identification

Relationships

- One Quiz → Many Quiz Attempts
- One Student → Many Quiz Attempts

Attributes

Column	Data Type	Key	Description
attempt_id	INT	PK	Unique quiz attempt identifier
quiz_id	INT	FK → Quiz(quiz_id)	Attempted quiz
student_id	INT	FK → Student(student_id)	Student attempting the quiz
score	DECIMAL(5,2)		Score obtained
attempted_at	DATETIME		Date and time of the attempt

Design Justification

Quiz attempts are stored separately to allow multiple students to attempt the same quiz while maintaining individual performance records. Each student can attempt a quiz only once. Therefore, the combination of **quiz_id** and **student_id** should remain unique.

Table 5.4 – WeeklySummary

Purpose

Stores a weekly summary of each student's learning activities, including topics covered, homework assigned, and areas requiring additional improvement.

Supported User Stories

- Story 5.3 – Weekly Summary Notification

Relationships

- One Tutor → Many Weekly Summaries
- One Student → Many Weekly Summaries

Attributes

Column	Data Type	Key	Description
summary_id	INT	PK	Unique weekly summary identifier
tutor_id	INT	FK → Tutor(tutor_id)	Tutor preparing the summary
student_id	INT	FK → Student(student_id)	Student receiving the summary
week_start	DATE		Beginning of the summary period
week_end	DATE		End of the summary period
topics_taught	TEXT		Topics covered during the week
homework_summary	TEXT		Summary of homework assigned
areas_for_improvement	TEXT		Areas requiring additional attention
created_at	DATETIME		Summary creation timestamp

Design Justification

Weekly summaries provide an overview of a student's learning across multiple sessions rather than focusing on individual classes. Keeping summaries separate improves parent communication and progress reporting.

6. Module 5 – Scheduling

The Scheduling module manages session bookings and virtual meetings. It enables students to book sessions and allows the tutor to schedule online meetings with students or parents when required.

Table 6.1 – SessionBooking

Purpose

Stores session bookings made by students. This table also represents the relationship between students and sessions, allowing multiple students to book the same regular session while supporting one-to-one sessions.

Supported User Stories

- Story 3.2 – Flexible Session Booking
- Story 3.4 – One-to-One Session Booking

Relationships

- One Session → Many Session Bookings
- One Student → Many Session Bookings

Attributes

Column	Data Type	Key	Description
booking_id	INT	PK	Unique booking identifier
session_id	INT	FK → Session(session_id)	Booked session
student_id	INT	FK → Student(student_id)	Student making the booking
booking_date	DATETIME		Date and time of booking
booking_status	ENUM		Pending, Confirmed, Cancelled

Design Justification

The SessionBooking table functions as both the booking record and the junction table between students and sessions, eliminating the need for a separate SessionStudent table. A student can book a particular session only once. Therefore, the combination of **session_id** and **student_id** should remain unique.

Table 6.2 – MeetingRequest**Purpose**

Stores virtual meeting requests scheduled by the tutor with either a parent or a student.

Supported User Stories

- Story 3.3 – Virtual Meeting Request

Relationships

- One Tutor → Many Meeting Requests
- One Parent → Many Meeting Requests
- One Student → Many Meeting Requests

Attributes

Column	Data Type	Key	Description
meeting_id	INT	PK	Unique meeting identifier
tutor_id	INT	FK → Tutor(tutor_id)	Tutor scheduling the meeting
student_id	INT	FK → Student(student_id), NULL	Student participant (if applicable)
parent_id	INT	FK → Parent(parent_id), NULL	Parent participant (if applicable)
meeting_date	DATETIME		Scheduled meeting date and time
meeting_link	VARCHAR(255))		Virtual meeting link
meeting_reason	VARCHAR(255))		Purpose of the meeting
status	ENUM		Scheduled, Completed, Cancelled

Design Justification

Meeting requests may involve either a student or a parent. Separate foreign keys keep these relationships explicit while maintaining a simple and understandable database structure. Exactly one of **student_id** or **parent_id** must contain a value for each meeting request.

7. Module 6 – Communication

The Communication module facilitates interaction between tutors, students, and parents through messaging, notifications, and frequently asked questions.

Table 7.1 – Message

Purpose

Stores messages exchanged between tutors, parents, and students through the LearnAtHome platform.

Supported User Stories

- Story 1.2 – Two-Way Parent-Tutor Messaging

Relationships

Messages can be exchanged between:

- Tutor ↔ Parent
- Tutor ↔ Student

Attributes

Column	Data Type	Key	Description
message_id	INT	PK	Unique message identifier
sender_type	ENUM		Tutor, Parent, Student
sender_id	INT		Sender identifier
receiver_type	ENUM		Tutor, Parent, Student
receiver_id	INT		Receiver identifier
subject	VARCHAR(150)		Message subject
message	TEXT		Message content
reply_message	TEXT		Reply content
sent_at	DATETIME		Time message was sent
replied_at	DATETIME		Time reply was sent

Design Justification

Since users are stored in separate tables, both the participant type and participant identifier are required to uniquely identify the sender and receiver.

Table 7.2 – FAQ

Purpose

Stores frequently asked questions and their corresponding answers to help students quickly resolve common queries.

Supported User Stories

- Story 1.3 – FAQ Board

Relationships

- This table has no foreign key relationships.

Attributes

Column	Data Type	Key	Description
faq_id	INT	PK	Unique FAQ identifier
question	VARCHAR(255)		Frequently asked question
answer	TEXT		Tutor's answer
category	VARCHAR(50)		Academic, Platform, General

Design Justification

FAQs are platform-wide resources rather than user-specific records. Keeping them separate enables centralized management of commonly asked questions.

Table 7.3 – Notification

Purpose

Stores notifications sent to students and parents regarding session updates, reminders, quizzes, assignments, and weekly summaries.

Supported User Stories

- Story 1.1 – In-App Session Update Notification
- Story 3.1 – Unified Class Schedule Management (Reminders)
- Story 5.3 – Weekly Summary Notification

Relationships

- One Student → Many Notifications
- One Parent → Many Notifications

Attributes

Column	Data Type	Key	Description
notification_id	INT	PK	Unique notification identifier
recipient_type	ENUM		Parent, Student
recipient_id	INT		Recipient identifier
title	VARCHAR(150)		Notification title
message	TEXT		Notification content
notification_type	ENUM		Session Update, Reminder, Weekly Summary
is_read	BOOLEAN		Read status
created_at	DATETIME		Notification creation timestamp

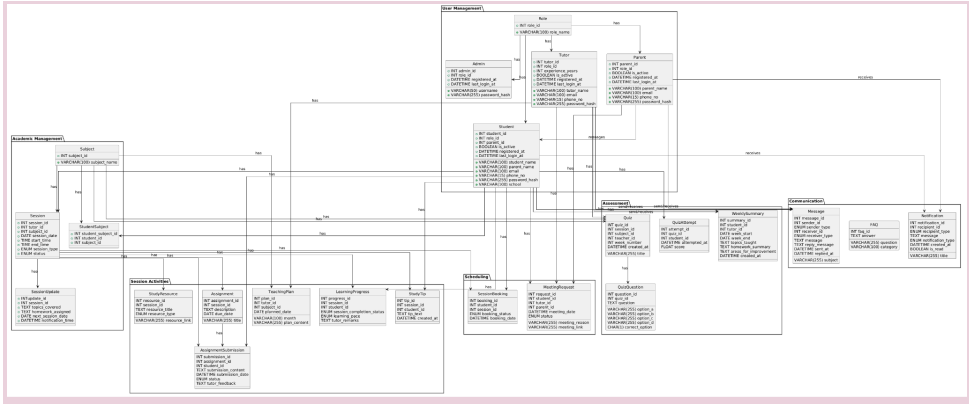
Design Justification

Notifications are maintained separately from messages because they represent system-generated updates rather than direct user communication.

8. Summary of Tables

Module	Tables
User Management	Roles, Admin, Tutor, Parent, Student
Academic Management	Subject, StudentSubject, Session, SessionUpdate
Session Activities	Assignment, AssignmentSubmission, LearningProgress, StudyTip, StudyResource, TeachingPlan
Assessment	Quiz, QuizQuestion, QuizAttempt, WeeklySummary
Scheduling	SessionBooking, MeetingRequest
Communication	Message, FAQ, Notification

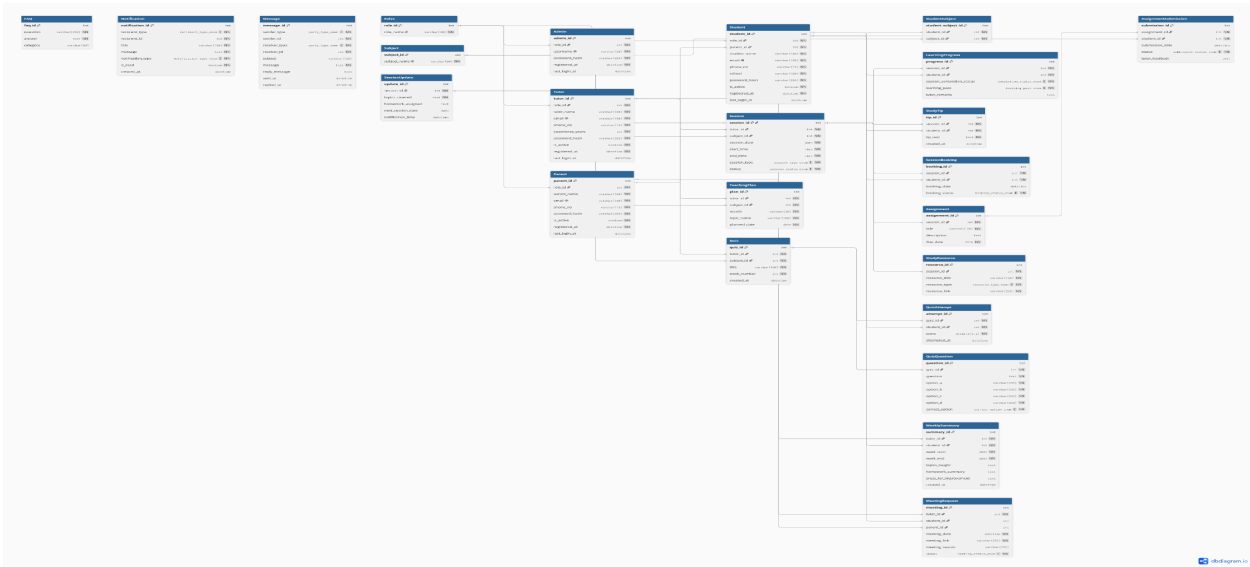
Class Diagram



[Click here to access the Class Diagram](#) (for clear view)

ER Diagram

The ER diagram illustrates the relational database structure of the LearnAtHome platform. It depicts the entities, primary keys, foreign key relationships, and associations among the six functional modules: User Management, Academic Management, Session Activities, Assessment, Scheduling, and Communication. The design follows the finalized user stories and business rules established during the system analysis phase.



[Click here to access the ER diagram](#) (for clear view)

Sprint Schedule

- ❖ **Sprint 1: Milestone 1 – Identify User Requirements** (18/06/2026 - 26/06/2026)
 - Identify primary, secondary, and tertiary users of the application.
 - Design open-ended interview/survey questions focused on users' problems and challenges (not feature suggestions).
 - Collect audio/video evidence.
 - Write SMART user stories: As a [user], I want [action], so that [benefit]. Review and refine the stories
 - Compile the Milestone 1 deliverables (user identification, interview findings, pain point analysis, user stories, and supporting interview proof into the final PDF report.)

- ❖ **Sprint 2: Milestone 2 – Scheduling and Design** (28/06/2026 - 10/07/2026)
 - Prepare the overall project schedule based on the user stories identified in Milestone 1.
 - Finalize the sprint timeline and task allocation.
 - Design the system architecture, class diagram, and database schema. (ER diagram)
 - Setup/Maintain Jira, Gantt chart and Kanban board for task scheduling/allocations.
 - Design and develop the frontend pages as per the planned theme and workflow.
 - Conduct scrum meetings, record minutes, and review progress. Compile all deliverables for the Milestone 2 report.

- ❖ **Sprint 3: API Development & Integration [Sprint 1]** (20/07/2026 - 28/07/2026)
 - Finalize the API endpoints required for the selected user stories.
 - Develop backend APIs and integrate the required external services.
 - Prepare a Swagger-compatible YAML file documenting all APIs.
 - Implement input validation, error handling.
 - Maintain user story mapping for API.
 - Write and execute API test cases along with basic **pytest** unit tests.

- ❖ **Sprint 4: Backend Integration and Testing [Sprint 2]** (29/07/2026 - 05/08/2026)
 - Integrate the frontend with the developed backend APIs.
 - Test key workflows, including tutor management, student enrollment, and session booking.
 - Execute API, functional, and unit (**pytest**) test cases.
 - Resolve integration issues and functional bugs.
 - Collect user feedback and document the findings.
 - Update the project documentation, README, and overall project progress.

❖ **Sprint 5: Final Review & Project Refinement** (05/08/2026 - 09/08/2026)

- Complete end-to-end testing of the application.
- Resolve pending issues and verify all user stories.
- Review code quality and update project documentation.
- Compile the final project report by consolidating all milestones and sprint deliverables.
- Create the presentation slides and record the project demonstration.

❖ **Sprint 6: Release Preparation & Final Submission** (09/08/2026 - 10/08/2026)

- Conduct the final quality review of the application.
- Complete the project documentation and final update README.
- Finalize the presentation and demonstration video.
- Prepare the submission package(zip) with source code and reports.
- Submit all project deliverables.

SCRUM Meetings Schedule and Minutes

Weekly SCRUM meetings held via Google Meet to review progress, resolve issues, and plan upcoming tasks.

Minutes from 18-06-2026 SCRUM Meeting

- Conducted team introductions and discussed individual roles and collaboration.
- Reviewed the project guidelines, objectives, and expected deliverables.
- Discussed the project workflow, milestones, and submission timeline.
- Planned the next meeting to finalize the project domain.

Minutes from 22-06-2026 SCRUM Meeting

- Reviewed the feedback and suggestions shared during the faculty session.
- Explored multiple project domains and finalized the **Educational Domain**.
- Discussed the initial problem statement, user roles, and platform workflow.
- Planned user research activities and SMART user story preparation for Milestone 1.

Minutes from 27-06-2026 SCRUM Meeting

- Reviewed the feedback received during the TA session for Milestone 1.
- Verified user identification, interview findings, pain points, and **SMART** user stories.
- Finalized the documentation, formatting, supporting evidence, and completed the final review of Milestone 1 deliverables.
- Discussed potential platform features and workflows derived from the finalized user stories to guide Milestone 2.

Minutes from 30-06-2026 SCRUM Meeting

- Reviewed the Milestone 2 guidelines and expected deliverables.
- Discussed the frontend development plan and dashboard allocation.
- Finalized the initial database design and system architecture.
- Planned sprint schedules, Scrum meetings, and updated the Kanban board.

Minutes from 03-07-2026 SCRUM Meeting

- Finalized UI theme, navigation flow, and wireframe planning.
- Discussed user role-wise page distribution and team responsibilities.
- Worked on the database schema, ER diagram and required system components.
- Updated the sprint plan, Gantt chart, and project schedule.

Minutes from 05-07-2026 SCRUM Meeting

- Reviewed the ER diagram, database schema, and class diagram.
- Discussed frontend implementation progress and README structure.
- Prepared the initial draft of the Milestone 2 documentation.
- Finalized the frontend technology stack(Vue 3, tailwind etc) and UI wireframes.

Minutes from 08-07-2026 SCRUM Meeting

- Reviewed the completed frontend pages and navigation flow.
- Refined the Admin and Tutor dashboards based on team feedback.
- Updated the Kanban board with completed and pending tasks.
- Assigned the remaining frontend development tasks and sprint plannings.

Minutes from 10-07-2026 SCRUM Meeting

- Reviewed the feedback received during the TA/client meeting.
- Planned the implementation of AI-based features and remaining dashboards.
- Discussed bug fixes and frontend improvements.
- Reviewed the Milestone 2 report and sprint progress.

Minutes from 15-07-2026 SCRUM Meeting

- Verified all frontend pages and completed UI refinements.
- Reviewed the database design, diagrams, and documentation.
- Updated the README, Kanban board, and Gantt chart.
- Finalized the Milestone 2 report and supporting deliverables.

Minutes from 21-07-2026 SCRUM Meeting

- Conducted the final verification of Milestone 2 deliverables.
- Reviewed the frontend implementation and project documentation.
- Confirmed that all diagrams, schedules, and reports were complete.
- Planned the activities and task allocation for the next milestone.

Project Planning & Progress Tracking

To effectively plan, organize, and monitor project activities, the team adopted an Agile-based workflow supported by a **Gantt Chart**, **SCRUM meetings**, and a **Kanban Board**. These tools helped ensure proper task planning, collaboration, and timely completion of each milestone.

❖ **Gantt Chart**

- Created to define the overall project timeline, sprint schedule, and milestone deadlines.
- Used to monitor task dependencies and keep the project on track.

❖ **SCRUM Meetings**

- Conducted regularly to review progress, discuss challenges, and plan upcoming sprint activities.
- Helped the team monitor deliverables, assign responsibilities, and resolve blockers.

❖ **Kanban Board**

- Used to create, assign, prioritize, and track project tasks.
- Tasks were organized into **To Do**, **In Progress**, **Review**, and **Completed** stages.

❖ **Team Collaboration**

- Responsibilities were distributed among team members based on project requirements.
- Progress was reviewed and updated during each sprint meeting.

❖ **Progress Monitoring**

- Project activities were continuously tracked to ensure milestones were completed on schedule.
- Regular reviews helped identify pending tasks and maintain steady project progress.

Outcome

- Improved coordination, enhanced task visibility, and ensured the timely completion of project deliverables.

Project Gantt chart

The **Gantt chart** provides a visual representation of the project schedule, illustrating the timeline of major activities, milestones, and sprints.

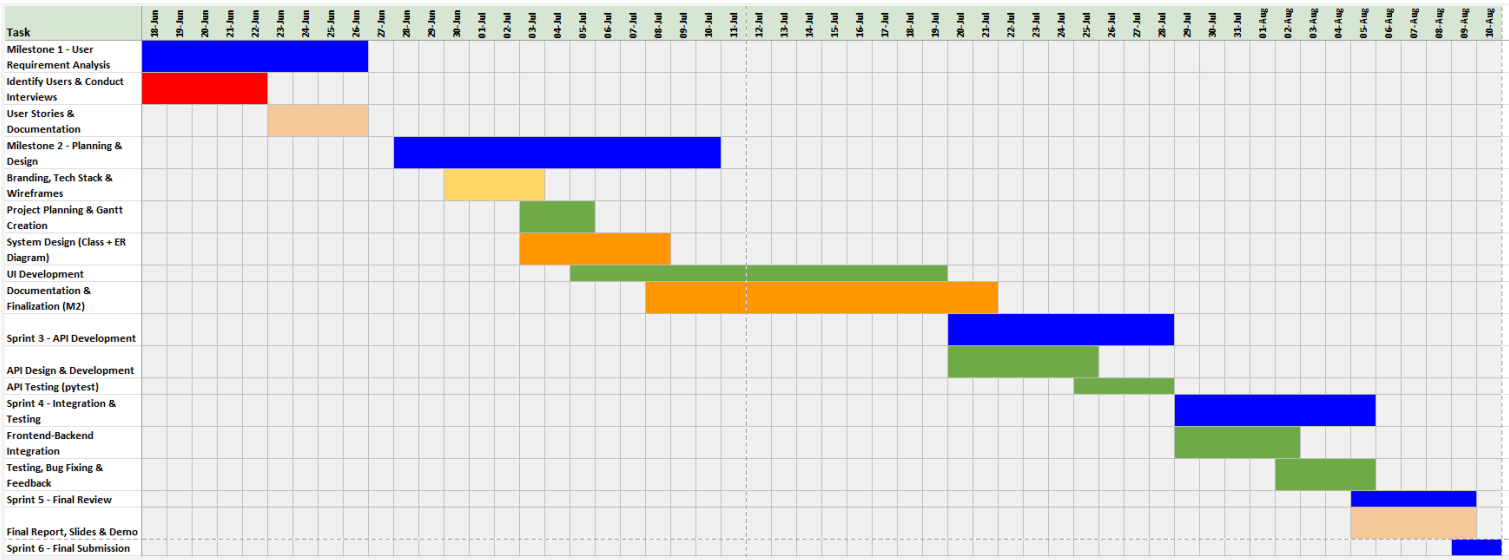
The project is divided into six phases, each focusing on a specific set of objectives:

- **Milestone 1:** Identify users, conduct interviews, analyze pain points, and prepare SMART user stories.
- **Milestone 2:** Plan the project schedule, design the system architecture, database schema, class diagrams, and develop the frontend interface.
- **Sprint 3:** Develop backend APIs, prepare Swagger-compatible YAML documentation, and perform API testing.
- **Sprint 4:** Integrate the frontend with backend APIs, execute testing, resolve issues, and incorporate user feedback.
- **Sprint 5:** Perform end-to-end testing, review project quality, finalize documentation, and prepare the project presentation.
- **Sprint 6:** Complete the final review, package the project files, and submit all required deliverables.

Note : For a more comprehensive breakdown of project activities, task dependencies, and timelines, the **detailed Gantt chart and Task Table** are [available here](#).

Task Table			
Task Table			
Task	Start Date	End Date	
Milestone 1 – User Requirement Analysis	18-Jun-2026	26-Jun-2026	
User Research & SMART User Stories	20-Jun-2026	26-Jun-2026	
Milestone 2 – Project Planning & Design	28-Jun-2026	10-Jul-2026	
System Design (Architecture, ER & Class Diagram)	30-Jun-2026	08-Jul-2026	
Frontend UI Development	01-Jul-2026	15-Jul-2026	
Sprint Planning, Gantt & Kanban Setup	30-Jun-2026	10-Jul-2026	
Sprint 3 – API Development & Integration	20-Jul-2026	28-Jul-2026	
Sprint 4 – Backend Integration & Testing	29-Jul-2026	05-Aug-2026	
Sprint 5 – Final Review & Project Refinement	05-Aug-2026	09-Aug-2026	
Sprint 6 – Final Check	09-Aug-2026	10-Aug-2026	

Task Table



Gantt Chart

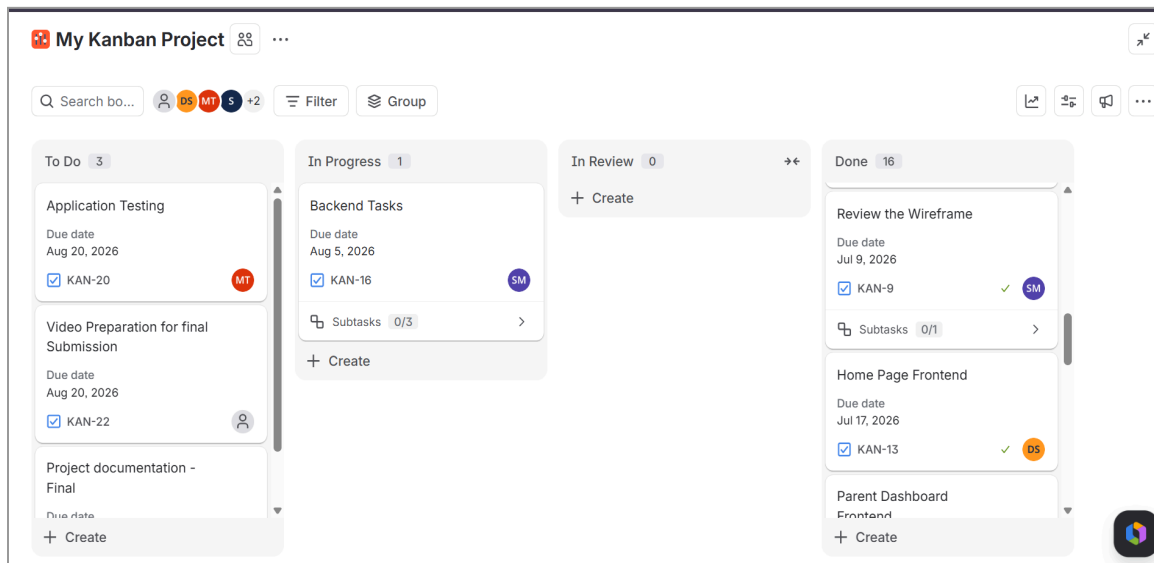
PROJECT TASK SCHEDULE			
PHASE	TASK / ACTIVITY	START DATE	END DATE
Milestone 1	Identify Primary, Secondary & Tertiary Users	18-Jun-2026	19-Jun-2026
Milestone 1	Conduct User Interviews & Collect Evidence	19-Jun-2026	22-Jun-2026
Milestone 1	Analyze Pain Points	22-Jun-2026	23-Jun-2026
Milestone 1	Write SMART User Stories	23-Jun-2026	24-Jun-2026
Milestone 1	Compile & Submit Milestone 1 Report	25-Jun-2026	26-Jun-2026
Milestone 2	Project Planning & Overall Schedule	28-Jun-2026	29-Jun-2026
Milestone 2	System Architecture Design	30-Jun-2026	02-Jul-2026
Milestone 2	Database Design (ER Diagram & Schema)	30-Jun-2026	06-Jul-2026
Milestone 2	Class Diagram & System Modeling	03-Jul-2026	08-Jul-2026
Milestone 2	Frontend UI Design & Development	01-Jul-2026	15-Jul-2026
Milestone 2	Setup, Maintain Jira, Commit to GitHub	26-Jun-2026	08-Aug-2026
Sprint 3	Sprint 3 – API Development & Integration	20-Jul-2026	28-Jul-2026
Sprint 4	Sprint 4 – Backend Integration & Testing	29-Jul-2026	05-Aug-2026
Sprint 5	Sprint 5 – Final Review & Project Refinement	05-Aug-2026	09-Aug-2026
Sprint 6	Sprint 6 – Release Preparation & Final Submission	09-Aug-2026	10-Aug-2026

Project Task Schedule

Project Task Management using Kanban Board

The project was managed using a **Kanban board**, where tasks were organized based on their current status, such as To Do, In Progress, Review, and Completed. The board was regularly updated during Scrum meetings to track sprint progress, assign new tasks, resolve pending issues, and maintain transparency among team members.

Below are some snapshots of Kanboard board/list.



Kanban Board

Work	Assignee	Reporter	Priority	Status	Unres
☐ KAN-14 Parent Dashboard Frontend	DS Drishya Shrivastava	SM Sreelakshmy J ...	Medium	TO DO	Unres
☐ KAN-15 Student Dashboard Frontend	SV Sudharshini V	SM Sreelakshmy J ...	Medium	TO DO	Unres
☐ KAN-16 Backend Tasks	SM Sreelakshmy J Menon	SM Sreelakshmy J ...	Medium	TO DO	Unres
☐ KAN-20 Application Testing	MT Medhavi Tripathi	SM Sreelakshmy J ...	Medium	TO DO	Unres
☐ KAN-21 MOM(client meeting) Document upload + Sen...	DS Drishya Shrivastava	SM Sreelakshmy J ...	Medium	TO DO	Unres
☐ KAN-22 Video Preparation for final Submission	Unassigned	SM Sreelakshmy J ...	Medium	TO DO	Unres

Kanban Project List Snapshot

Spaces

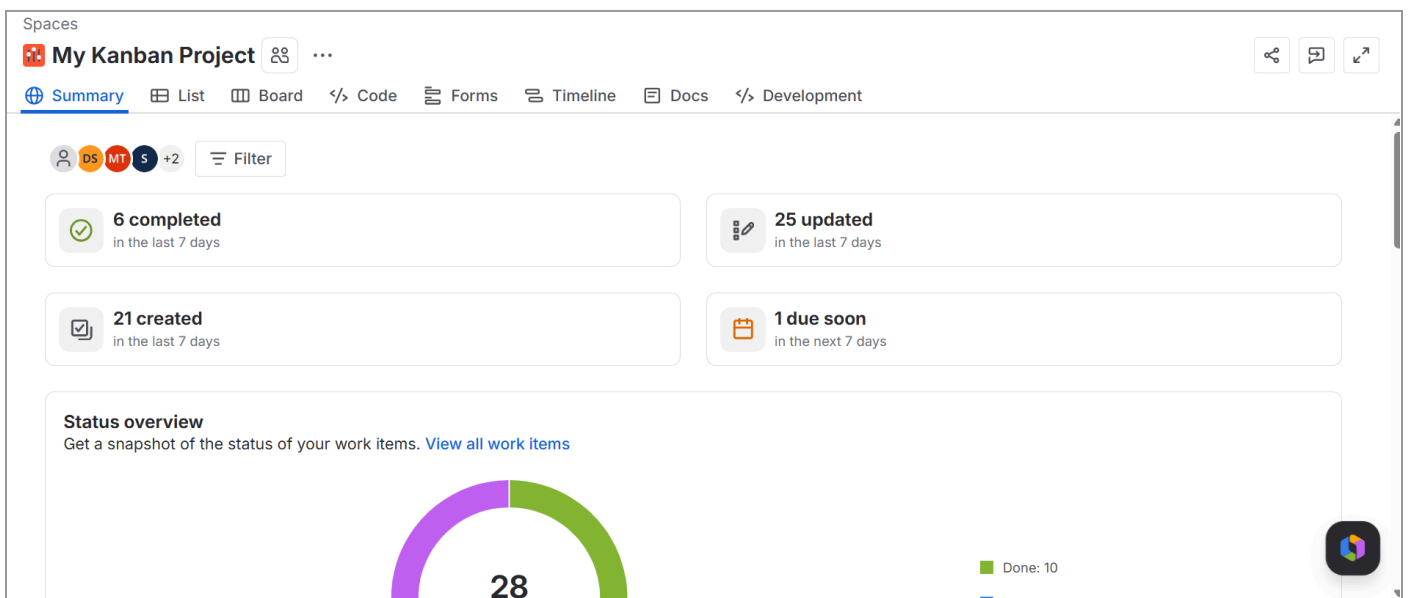
My Kanban Project ...

Summary List Board Code Forms Timeline Docs Development

Ask AI Search wo... Filter Group

	Work	Assignee	Reporter	Priority	Status	
☐	☒ KAN-4 Wireframe creation	SHIKHA	Drishya Shrivast...	Medium	DONE	Done
☐	☒ KAN-12 Tutor Dashboard Frontend	SHIKHA	Sreelakshmy J ...	Medium	IN PROGRESS	Unres
☐	☒ KAN-10 Admin Dashboard Frontend	Sudharshini V	Sreelakshmy J ...	Medium	IN PROGRESS	Unres
☐	☒ KAN-25 Database Schema	Medhavi Tripathi	Medhavi Tripathi	Medium	IN PROGRESS	Unres
☐	☒ KAN-2 ER Diagram	Medhavi Tripathi	Drishya Shrivast...	Medium	IN PROGRESS	Unres
☐	☒ KAN-8 Milestone 2 Document preparation	Sreelakshmy J Menon	Sreelakshmy J ...	Medium	IN PROGRESS	Unres

Kanban Project List snapshot



Kanban Project Summary

FrontEnd Pages ScreenShot

Frontend pages screenshots are uploaded in pdf format in the milestone 2 folder within the team's project folder.

[Click here to view Frontend Pages Screenshot](#)

-----END OF REPORT-----